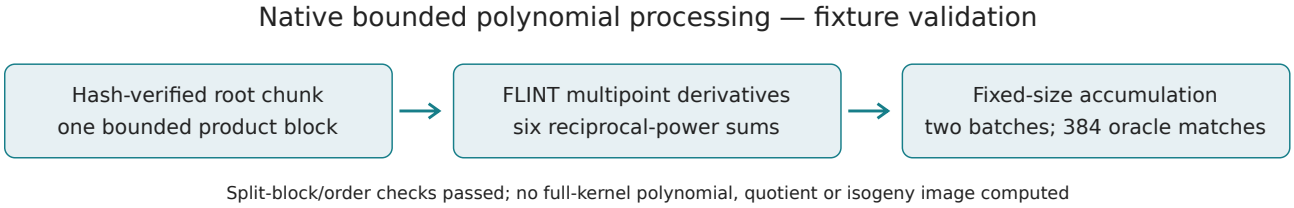
Native bounded polynomial-block stage

1 November 2026. The native stage prepares a monic polynomial block from one hash-verified root chunk and streams six reciprocal-power sums at a bounded vector of point contexts. It retains one block at a time, rather than allocating the global baby polynomial.



Correctness evidence

The root input is the previously independently verified 4,096-root S3 fixture for the frozen 80-bit kernel. Its hash-matched materialization came from the restored producer snapshot; storage transport and arithmetic validation remain separate. The field/kernel identity is unchanged.

All 192 sums (32 contexts × six powers) matched direct scalar inversion over every root. Two 2,048-root blocks, in either order, reproduced the single-block result. PARI separately verified each x([s]G), s = 1..32, the recorded generator's q-order and its non-membership in the ell-kernel. These are source-point contexts, not transported isogeny images.

A second batch checks the actual first 32 giant abscissae x([(4i+2)B]K), i=0..31, with B=427154957034. PARI independently verifies all giant contexts. All 192 reciprocal sums agree with direct inversion and split-block evaluation. The two batches therefore compare 384 values; both still use only the 4,096-root baby prefix, not the full B-root window.

Nine negative controls reject truncated or malformed blocks, wrong field, oversize degree, nonmonicity, extra data, a root collision, noncanonical contexts and too many contexts. Original native trace-helper checks also pass. No elapsed-time or peak-RSS measurement was taken.

Prepared data and storage

The serialized monic degree-4,096 product occupies 131,148 bytes, with magic, degree and field header plus canonical fixed-width coefficients. It is selected for trusted S3 upload/full-byte readback. A pointer commit and storage receipt establish completed migration; the creation-time manifest does not claim it. The low-level evaluator sums provided blocks with multiplicity; a future controller must certify complete, nonoverlapping ordinal coverage.

Scope and next integration

This is a native development fixture, not a completed 80-bit map, globally fast external-memory tree or an end-to-end feasibility result. Many blocks and point batches can still require excessive total computation and I/O. Integrate actual giant-point batches with the codomain moment formulas, then implement the full point map. Price setup, serialization, storage, hydration, arithmetic and verification before any performance claim.

Source: `../native/poly_blocks.c`; raw direct/streamed outputs, rejection records, context checks and hashes: `../poly-blocks/`. Build/run contracts and S3 retrieval are in that directory's README. The full acceptance criterion remains in `../MEASUREMENT_PROTOCOL.md`.

This implements the existing CM constructor's reciprocal-power recurrence and FLINT root-product/fast-multipoint API contracts. It asserts no new algebraic theorem, admitted benchmark comparison or crossing record.